

How to make this "Prerequisites Excel add-in development environment set up. Office.js library included in your project. A clean Excel file loaded in Excel."

Setup Environment steps:

To set up an Excel add-in development environment and include the Office.js library, follow these steps:

Prerequisites for Excel Add-in Development

1. Set Up Excel Add-in Development Environment

- **Install Node.js:**
 - Download and install [Node.js](#) (LTS version recommended). Node.js comes with npm (Node Package Manager), which is essential for managing packages.
- **Install Yeoman and Office Add-in Generator:**
 - Open a command prompt or terminal and run the following command:

bash

```
npm install -g yo generator-office
```

- **Create a New Excel Add-in Project:**
 - Navigate to the directory where you want to create your project and run:

bash

```
yo office
```

- You will be prompted to choose a project type. Select **"Excel"** and choose from the available frameworks (e.g., React, Angular, or just HTML/JavaScript).

2. Office.js Library Included in Your Project

- The Office.js library is included automatically when you create a new project using the Yeoman generator. You can verify this by looking for the inclusion of the script in the index.html file (or similar):

html

```
<script  
src="https://appsforoffice.microsoft.com/lib/1/hosted/office.debug.js  
></script>
```

-
-
-

- In production, you might want to use the minified version:

html

```
<script  
src="https://appsforoffice.microsoft.com/lib/1/hosted/office.js"></sc  
ript>
```

3. Build the Project

- Navigate to your project folder in the command prompt or terminal and run:

bash

```
npm install
```

This installs all necessary dependencies.

- Next, run :

bash

```
npm start
```

This command will start a local web server and host your add-in.

- This will usually open Excel or, if not open, you can manually open Excel, go to **Insert > Add-ins > My Add-ins**, and select **"Shared Folder"** or enter the URL of your local server.

4. Clean Excel File Loaded in Excel

- Open Excel and create a new workbook or use an existing clean Excel file. Make sure it doesn't contain any unnecessary data or formatting that might interfere with your tests.
- Save the file in an easily accessible location so you can later interact with it through your add-in.

Summary

By following these steps, you will set up an Excel add-in development environment with the Office.js library included. You will have a clean Excel file ready for development and testing. This setup allows you to start implementing your scripts and test them in Excel seamlessly.

Test:

Use this instructions document named "Excel_SFT_Changelog_sample" and solve this test "Welcome to the Office.JS Proficiency Test". In this test, you will receive 2 Excel files (a clean file and a final file) as well as a change log. Your task is to write a script that will transform the clean file into the final file using Office.JS JavaScript. You will copy/paste your JavaScript into this form for evaluation." using these two files "Excel_SFT_input_file_sample" and "Excel_SFT_final_file_sample".

Solution:

To transform the "Excel_SFT_input_file_sample" into the "Excel_SFT_final_file_sample" using Office.js, I'll guide you through the key steps outlined in the change log and provide a script that implements these transformations.

Overview of the Task

1. **Read Raw Data** from the input file.
2. **Calculate Total Revenue, Weighted Average Margin, Rolling 3-Month Trend, Year-over-Year Margin Delta, and Margin Health** using the specified formulas.
3. **Set up a Chart Data Table** for visualization.
4. **Create a Combo Chart** in the final output.

JavaScript Code

Here's the script that you can use:

```
javascript
// Ensure Office is ready
Office.onReady((info) => {
    if (info.host === Office.HostType.Excel) {
        document.getElementById("run").onclick = runTransform;
    }
});

async function runTransform() {
    await Excel.run(async (context) => {
        const inputSheet =
context.workbook.worksheets.getActiveWorksheet();
        const dashboardSheet =
context.workbook.worksheets.add("Dashboard");

        // Step 1: Load Raw Data
        const rawDataRange = inputSheet.getRange("A1:G187");
        rawDataRange.load("values");
        await context.sync();

        // Step 2: Prepare Dashboard headers
        const headers = [
            "Product", "Quarter", "Total Revenue", "Weighted Avg Margin",
            "Rolling 3-Mo Trend", "YoY Margin Delta", "Margin Health"
```

```

];
dashboardSheet.getRange("A7:G7").values = [headers];

const products = ["Widget Pro", "Widget Standard", "Service
Package", "Accessory Kit"];
const quarters = ["2023 Q1", "2023 Q2", "2023 Q3", "2023 Q4", "2024
Q1", "2024 Q2", "2024 Q3", "2024 Q4"];

let rowIndex = 8; // Starting row for data
for (const product of products) {
  for (const quarter of quarters) {
    // Calculate Total Revenue
    const totalRevenueFormula = `SUMIFS('Raw
Data'!$E$2:$E$187, 'Raw Data'!$D$2:$D$187, "${product}", 'Raw
Data'!$B$2:$B$187, VALUE(LEFT("${quarter}", 4)), 'Raw
Data'!$C$2:$C$187, RIGHT("${quarter}", 2))`;
    dashboardSheet.getRange(`C${rowIndex}`).formulas =
[[totalRevenueFormula]];

    // Calculate Weighted Avg Margin
    const weightedAvgMarginFormula = `SUMPRODUCT(('Raw
Data'!$D$2:$D$187="${product}") * ('Raw
Data'!$B$2:$B$187=VALUE(LEFT("${quarter}", 4))) * ('Raw
Data'!$C$2:$C$187=RIGHT("${quarter}", 2)) * 'Raw
Data'!$G$2:$G$187) / C${rowIndex}`;
    dashboardSheet.getRange(`D${rowIndex}`).formulas =
[[weightedAvgMarginFormula]];

    // Calculate Rolling 3-Mo Trend
    const rolling3MoTrendFormula =
`IF(AND(LEFT("${quarter}", 4)="2023", RIGHT("${quarter}", 2)="Q1"), "N/A",
D${rowIndex}-D${rowIndex-1})`;
    dashboardSheet.getRange(`E${rowIndex}`).formulas =
[[rolling3MoTrendFormula]];

    // Calculate YoY Margin Delta
    const yoyMarginDeltaFormula =
`IF(LEFT("${quarter}", 4)="2023", "N/A", D${rowIndex}-INDEX($D$8:$D$39,
MATCH(${product}&LEFT("${quarter}", 4)-
1&RIGHT("${quarter}", 2), $A$8:$A$39&LEFT($B$8:$B$39, 4)&RIGHT($B$8:$B$39, 2), 0
)))`;
    dashboardSheet.getRange(`F${rowIndex}`).formulas =
[[yoyMarginDeltaFormula]];

    // Calculate Margin Health
    const marginHealthFormula = `IF(D${rowIndex}>0.35,
"Strong", IF(D${rowIndex}>=0.2, "Moderate", "At Risk"))`;
    dashboardSheet.getRange(`G${rowIndex}`).formulas =
[[marginHealthFormula]];

    rowIndex++;
  }
}

await context.sync();

// Optional: Format cells for the final dashboard
dashboardSheet.getRange("C8:C39").numberFormat = [["$#", "#0"]];
dashboardSheet.getRange("D8:D39").numberFormat = [["0.00%"]];
dashboardSheet.getRange("E8:E39").numberFormat = [["0.00%"]];
dashboardSheet.getRange("F8:F39").numberFormat = [["0.00%"]];

```

```

    // Step 3: Set up Chart Data Table
    const chartDataHeader = ["Quarter", "Widget Pro", "Widget
Standard", "Service Package", "Accessory Kit", "Total Revenue"];
    dashboardSheet.getRange("A42:F42").values = [chartDataHeader];

    const chartRowStart = 43;
    for (let i = 0; i < quarters.length; i++) {
        dashboardSheet.getRange(`A${chartRowStart + i}`).values =
[[quarters[i]]];
    }

    await context.sync();

    // Step 4: Add Chart Data Formulas
    for (let i = 0; i < quarters.length; i++) {
        dashboardSheet.getRange(`B${chartRowStart + i}`).formulas =
[[`SUMPRODUCT(($A$8:$A$39="Widget
Pro")*($B$8:$B$39=${quarters[i]})*($D$8:$D$39))`]];
        dashboardSheet.getRange(`C${chartRowStart + i}`).formulas =
[[`SUMPRODUCT(($A$8:$A$39="Widget
Standard")*($B$8:$B$39=${quarters[i]})*($D$8:$D$39))`]];
        dashboardSheet.getRange(`D${chartRowStart + i}`).formulas =
[[`SUMPRODUCT(($A$8:$A$39="Service
Package")*($B$8:$B$39=${quarters[i]})*($D$8:$D$39))`]];
        dashboardSheet.getRange(`E${chartRowStart + i}`).formulas =
[[`SUMPRODUCT(($A$8:$A$39="Accessory
Kit")*($B$8:$B$39=${quarters[i]})*($D$8:$D$39))`]];
        dashboardSheet.getRange(`F${chartRowStart + i}`).formulas =
[[`SUMIF($B$8:$B$39,$A${chartRowStart + i},$C$8:$C$39)`]];
    }

    await context.sync();

    // Step 5: Create the Chart
    const chartDataRange = dashboardSheet.getRange("A42:F51");
    const chart =
dashboardSheet.charts.add(Excel.ChartType.columnClustered, chartDataRange,
Excel.ChartSeriesBy.auto);

    // Configure chart
    chart.setTitle("Quarterly Margin Trends by Product");
    chart.setLegendPosition(Excel.ChartLegendPosition.bottom);

    await context.sync();
}).catch((error) => {
    console.error(error);
    if (error instanceof OfficeExtension.Error) {
        console.error("Debug info: " +
JSON.stringify(error.debugInfo));
    }
});
}

```

Explanation of the Script

1. **Setup Office.js:**
 - The script starts by checking if Office is ready and binds the `runTransform` function to a button click.
2. **Read Raw Data:**
 - The script reads from the **Raw Data** tab and loads the relevant data into the context.
3. **Calculate Metrics:**
 - For each product and quarter, it calculates Total Revenue, Weighted Avg Margin, Rolling 3-Mo Trend, and YoY Margin Delta using the formulas specified.
4. **Setup Chart Data:**
 - It sets up a table for the chart data with totals for each product per quarter. It uses `SUMPRODUCT` to pull the relevant data from the calculated metrics.
5. **Create the Chart:**
 - Finally, it creates a clustered column chart with the relevant data and adds a title.

Running the Script

To run this script:

- Ensure you have an Excel add-in set up properly.
- Place the script in the JavaScript section of your add-in.
- Trigger the `runTransform` function via a button in your add-in user interface.

This implementation should effectively transform the clean input file into the specified final output format as required by the proficiency test!